МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №4

з дисципліни «Алгоритми та структури даних»

Виконав:

студент групи ІМ-о51

Машута О.І

Перевірив:

ас. Череватенко О.В.

Київ 2026

Постановка задачі 1. Представити напрямлений та ненапрямлений графи із заданими пара метрами так само, як у лабораторній роботі №3. Відмінність: коефіцієнт k 1.0 n3 0.01 n4 0.01 0.3. Отже, матриця суміжності Adir напрямленого графа за варіантом фор мується таким чином: 1) встановлюється параметр (seed) генератора випадкових чисел, рів не номеру варіанту n1n2n3n4; 2) матриця розміром n n заповнюється згенерованими випадковими числами в діапазоні [0, 2.0); 3) обчислюється коефіцієнт k 1.0 n3 0.01 n4 0.01 0.3, кожен елемент матриці множиться на коефіцієнт k; 4) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0. 2. Обчислити: 1) степені вершин напрямленого і ненапрямленого графів; 2) напівстепені виходу та заходу напрямленого графа; 3) чи є граф однорідним (регулярним), і якщо так, вказати степінь однорідності графа; 42 4) перелік висячих та ізольованих вершин. Результати вивести у графічне вікно, консоль або файл. 3. Змінити матрицю Adir, коефіцієнт k 1.0 n3 0.005 n4 0.005 0.27. 4. Для нового орграфа обчислити: 1) півстепені вершин; 2) всі шляхи довжини 2 і 3; 3) матрицю досяжності; 4) матрицю сильної зв'язності; 5) перелік компонент сильної зв'язності; 6) граф конденсації. Результати вивести у графічне вікно, в консоль або файл. Шляхи довжиною 2 і 3 слід шукати за матрицями A2 і A3, відповід но. Як результат вивести перелік шляхів, включно з усіма проміжними вершинами, через які проходить шлях. Матрицю досяжності та компоненти сильної зв'язності слід шукати за допомогою операції транзитивного замикання. У переліку компонент слід вказати, які вершини належать до кожної компоненти. Граф конденсації вивести у графічне вікно. При проєктуванні програми слід врахувати наступне: 1) мова програмування обирається студентом самостійно; 2) графічне зображення усіх графів має формуватися програмою з тими жвимогами, як у ЛР №3; 3) всі графи, включно із графом конденсації, обов'язково зображувати у графічному вікні; 43 4) типи та структури даних для внутрішнього представлення всіх даних у програмі слід вибрати самостійно; 5) обчислення перелічених у завданні результатів має виконуватися роз робленою програмою (не вручну і не сторонніми засобами); 6) матриці, переліки степенів та маршрутів тощо можна виводити в гра фічне вікно або консоль — на розсуд студента; 7) упереліку знайдених шляхів треба вказувати не лише початок та кінець шляху, але й усі проміжні вершини, через які він проходить (наприклад, 1– 5– 3– 2).

Текст програми: using System; using System.Collections.Generic; using System.Drawing; using System.Drawing.Drawing2D; using System.Linq; using System.Windows.Forms;

namespace GraphLab4 { static class Program { [STAThread] static void Main() { Application.SetHighDpiMode(HighDpiMode.SystemAware); Application.EnableVisualStyles();

Application.SetCompatibleTextRenderingDefault(false); Application.Run(new Form1());
}}

```
public partial class Form1 : Form
{
    const int n1 = 3;
    const int n2 = 4;
    const int n3 = 1;
    const int n4 = 2;

    private int N;
    private int[,] matrixDir;
    private int[,] matrixUndir;
    private int[,] matrixCondensation;
    private List<List<int>> sccComponents;

    private RichTextBox txtOutput;
    private PictureBox pbGraph;
    private Button btnTask1_2;
    private Button btnTask3_4;
    private bool showCondensation = false;

    public Form1()
    {
        InitializeComponentCustom();
        N = 10 + n3;
    }

    private void RunTask1_2(object sender, EventArgs e)
    {
        showCondensation = false;
        txtOutput.Clear();
        txtOutput.AppendText($"=== TASK 1 & 2 ===\nVariant:
{n1}{n2}{n3}{n4}, N={N}\n");

        double k = 1.0 - (n3 * 0.01) - (n4 * 0.01) - 0.3;
        txtOutput.AppendText($"Coefficient K = {k:F3}\n");

        matrixDir = GenerateMatrix(k);
        matrixUndir = SymmetrizeMatrix(matrixDir);

        PrintMatrix("Adjacency Matrix (Directed):", matrixDir);
```

```
        PrintMatrix("Adjacency Matrix (Undirected):", matrixUndir);

        CalculateDegrees(matrixDir, matrixUndir);
        pbGraph.Invalidate();
    }

    private void RunTask3_4(object sender, EventArgs e)
    {
        showCondensation = false;
        txtOutput.Clear();
        txtOutput.AppendText($"=== TASK 3 & 4 ===\n");

        double k = 1.0 - (n3 * 0.005) - (n4 * 0.005) - 0.27;
        txtOutput.AppendText($"New coefficient K = {k:F3}\n");

        matrixDir = GenerateMatrix(k);
        PrintMatrix("New Adjacency Matrix:", matrixDir);

        CalculateDegreesDirectedOnly(matrixDir);
        FindPaths(matrixDir, 2);
        FindPaths(matrixDir, 3);

        int[,] reachability = GetReachabilityMatrix(matrixDir);
        PrintMatrix("Reachability Matrix:", reachability);

        int[,] strongConn =
GetStrongConnectivityMatrix(reachability);
        PrintMatrix("Strong Connectivity Matrix:", strongConn);

        sccComponents = FindComponents(strongConn);
        txtOutput.AppendText("\nStrongly Connected Components:\n");
        for (int i = 0; i < sccComponents.Count; i++)
        {
            txtOutput.AppendText($"K{i + 1}: {{ {string.Join(", ",
sccComponents[i].Select(v => v + 1))} }}\n");
        }

        matrixCondensation = BuildCondensationMatrix(matrixDir,
sccComponents);
        PrintMatrix("Condensation Graph Matrix:",
matrixCondensation);
```

```
        showCondensation = true;
        pbGraph.Invalidate();
        MessageBox.Show("The condensation graph is displayed in the
graphics window.");
    }

    private int[,] GenerateMatrix(double k)
    {
        int seed = int.Parse($"{n1}{n2}{n3}{n4}");
        Random rand = new Random(seed);
        int[,] mat = new int[N, N];

        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                double val = rand.NextDouble() * 2.0;
                val = val * k;
                mat[i, j] = (val >= 1.0) ? 1 : 0;
            }
        }
        return mat;
    }

    private int[,] SymmetrizeMatrix(int[,] mat)
    {
        int[,] res = new int[N, N];
        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                if (mat[i, j] == 1 || mat[j, i] == 1) res[i, j] = 1;
                else res[i, j] = 0;
            }
        }
        return res;
    }

    private void CalculateDegrees(int[,] dir, int[,] undir)
    {
        txtOutput.AppendText("\n--- Graph Characteristics ---\n");
        txtOutput.AppendText("Undirected graph:\n");
```

```
        int[] deg = new int[N];
        bool regular = true;
        for (int i = 0; i < N; i++)
        {
            deg[i] = GetRowSum(undir, i);
            txtOutput.AppendText($"V{i + 1}: deg={deg[i]}\n");
            if (i > 0 && deg[i] != deg[i - 1]) regular = false;
        }

        if (regular) txtOutput.AppendText($"Graph is regular, degree
{deg[0]}.\n");
        else txtOutput.AppendText("Graph is not regular.\n");

        txtOutput.AppendText("\nDirected graph:\n");
        List<int> isolated = new List<int>();
        List<int> hanging = new List<int>();

        for (int i = 0; i < N; i++)
        {
            int degOut = GetRowSum(dir, i);
            int degIn = GetColSum(dir, i);
            txtOutput.AppendText($"V{i + 1}: out={degOut},
in={degIn}\n");

            if (degOut == 0 && degIn == 0) isolated.Add(i + 1);
            if ((degOut == 1 && degIn == 0) || (degOut == 0 && degIn
== 1)) hanging.Add(i + 1);
        }
        txtOutput.AppendText($"Isolated vertices: {(isolated.Count >
0 ? string.Join(",", isolated) : "none")}\n");
        txtOutput.AppendText($"Pendant vertices: {(hanging.Count > 0
? string.Join(",", hanging) : "none")}\n");
    }

    private void CalculateDegreesDirectedOnly(int[,] mat)
    {
        txtOutput.AppendText("\nVertex semi-degrees (out/in):\n");
        for (int i = 0; i < N; i++)
            txtOutput.AppendText($"V{i + 1}: out={GetRowSum(mat,
i)}, in={GetColSum(mat, i)}\n");
    }
```

```
    private void FindPaths(int[,] mat, int len)
    {
        txtOutput.AppendText($"\n--- Paths of length {len} ---\n");
        if (len == 2)
        {
            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    for (int k = 0; k < N; k++)
                        if (mat[i, k] == 1 && mat[k, j] == 1)
                            txtOutput.AppendText($"{i + 1} -> {k +
1} -> {j + 1}\n");
        }
        else if (len == 3)
        {
            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    for (int k = 0; k < N; k++)
                        if (mat[i, k] == 1)
                            for (int m = 0; m < N; m++)
                                if (mat[k, m] == 1 && mat[m, j] ==
1)
                                    txtOutput.AppendText($"{i + 1} -
> {k + 1} -> {m + 1} -> {j + 1}\n");
        }
    }

    private int[,] GetReachabilityMatrix(int[,] adj)
    {
        int[,] R = (int[,])adj.Clone();
        for (int i = 0; i < N; i++) R[i, i] = 1;
        for (int k = 0; k < N; k++)
            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    if (R[i, k] == 1 && R[k, j] == 1) R[i, j] = 1;
        return R;
    }

    private int[,] GetStrongConnectivityMatrix(int[,] R)
    {
        int[,] S = new int[N, N];
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
```

```
                if (R[i, j] == 1 && R[j, i] == 1) S[i, j] = 1;
                else S[i, j] = 0;
        return S;
    }

    private List<List<int>> FindComponents(int[,] S)
    {
        bool[] visited = new bool[N];
        List<List<int>> components = new List<List<int>>();
        for (int i = 0; i < N; i++)
        {
            if (!visited[i])
            {
                List<int> comp = new List<int>();
                for (int j = 0; j < N; j++)
                {
                    if (S[i, j] == 1)
                    {
                        comp.Add(j);
                        visited[j] = true;
                    }
                }
                components.Add(comp);
            }
        }
        return components;
    }

    private int[,] BuildCondensationMatrix(int[,] adj,
List<List<int>> comps)
    {
        int size = comps.Count;
        int[,] cond = new int[size, size];
        for (int i = 0; i < size; i++)
        {
            for (int j = 0; j < size; j++)
            {
                if (i == j) continue;
                bool edgeExists = false;
                foreach (int u in comps[i])
                {
                    foreach (int v in comps[j])
```

```
                    {
                        if (adj[u, v] == 1) { edgeExists = true;
break; }
                    }
                    if (edgeExists) break;
                }
                cond[i, j] = edgeExists ? 1 : 0;
            }
        }
        return cond;
    }

    private int GetRowSum(int[,] mat, int row)
    {
        int sum = 0;
        for (int j = 0; j < N; j++) sum += mat[row, j];
        return sum;
    }
    private int GetColSum(int[,] mat, int col)
    {
        int sum = 0;
        for (int i = 0; i < N; i++) sum += mat[i, col];
        return sum;
    }
    private void PrintMatrix(string title, int[,] mat)
    {
        txtOutput.AppendText($"\n{title}\n");
        int rows = mat.GetLength(0);
        int cols = mat.GetLength(1);
        txtOutput.AppendText("   ");
        for (int j = 0; j < cols; j++) txtOutput.AppendText($"{j +
1,2} ");
        txtOutput.AppendText("\n");
        for (int i = 0; i < rows; i++)
        {
            txtOutput.AppendText($"{i + 1,2}|");
            for (int j = 0; j < cols; j++)
txtOutput.AppendText($"{mat[i, j],2} ");
            txtOutput.AppendText("\n");
        }
    }
```

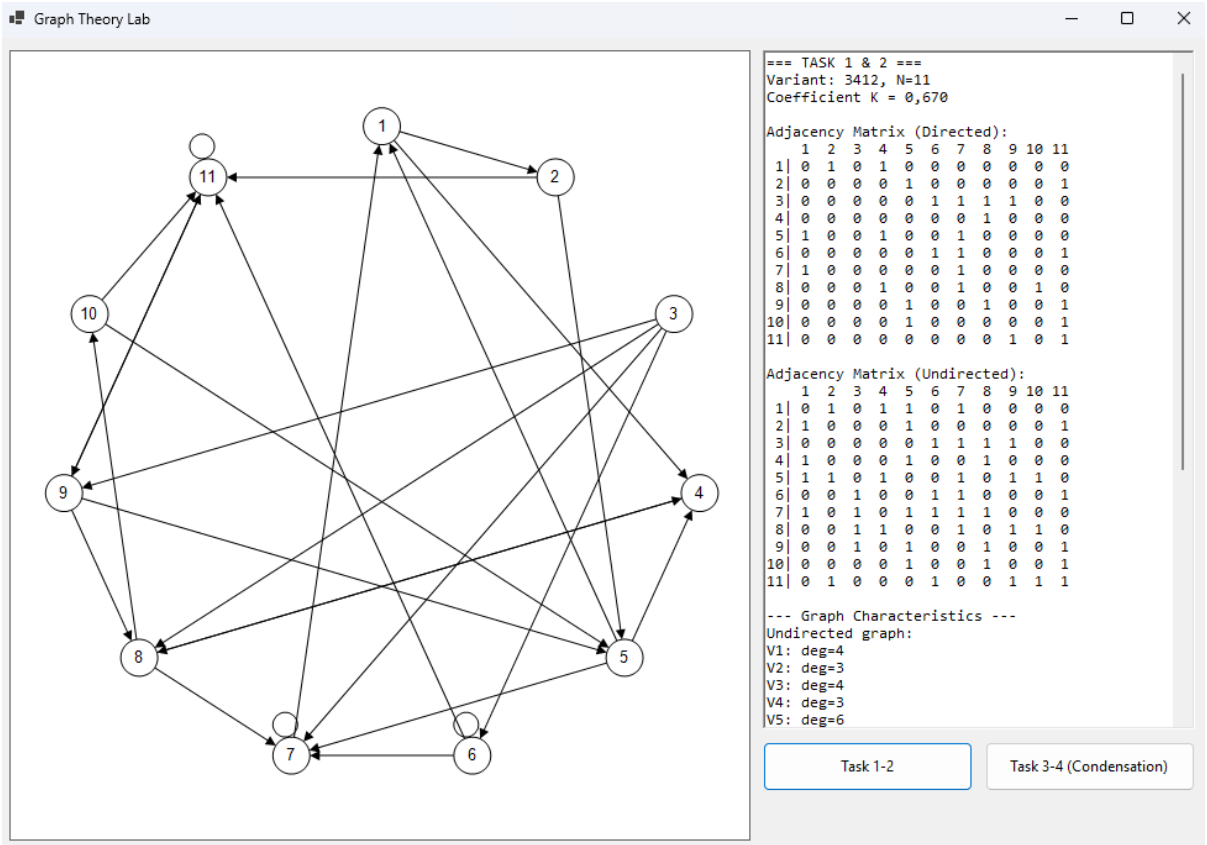
```
    private void pbGraph_Paint(object sender, PaintEventArgs e)
    {
        e.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
        if (matrixDir == null && !showCondensation) return;

        int w = pbGraph.Width;
        int h = pbGraph.Height;
        int centerX = w / 2;
        int centerY = h / 2;
        int radius = Math.Min(w, h) / 2 - 40;

        if (!showCondensation)
        {
            PointF[] points = GetCircleCoords(N, centerX, centerY,
radius);
            Pen p = new Pen(Color.Black, 1);
            p.CustomEndCap = new
System.Drawing.Drawing2D.AdjustableArrowCap(4, 4);

            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    if (matrixDir[i, j] == 1)
                    {
                        if (i == j) DrawLoop(e.Graphics, points[i]);
                        else DrawArrow(e.Graphics, p, points[i],
points[j]);
                    }
            DrawNodes(e.Graphics, points, Enumerable.Range(1,
N).Select(x => x.ToString()).ToArray());
        }
        else
        {
            if (matrixCondensation == null) return;
            int count = matrixCondensation.GetLength(0);
            PointF[] points = GetCircleCoords(count, centerX,
centerY, radius);
            Pen p = new Pen(Color.Blue, 2);
            p.CustomEndCap = new
System.Drawing.Drawing2D.AdjustableArrowCap(5, 5);

            for (int i = 0; i < count; i++)
                for (int j = 0; j < count; j++)
```

```
                    if (matrixCondensation[i, j] == 1)
                    {
                        if (i == j) DrawLoop(e.Graphics, points[i]);
                        else DrawArrow(e.Graphics, p, points[i],
points[j]);
                    }

            string[] labels = new string[count];
            for (int i = 0; i < count; i++) labels[i] = "K" + (i +
1);
            DrawNodes(e.Graphics, points, labels,
Brushes.LightGreen);
            e.Graphics.DrawString("Condensation Graph", new
Font("Arial", 14, FontStyle.Bold), Brushes.Blue, 10, 10);
        }
    }

    private PointF[] GetCircleCoords(int count, int cx, int cy, int
r)
    {
        PointF[] pts = new PointF[count];
        double angleStep = 2 * Math.PI / count;
        for (int i = 0; i < count; i++)
            pts[i] = new PointF((float)(cx + r * Math.Cos(i *
angleStep - Math.PI / 2)), (float)(cy + r * Math.Sin(i * angleStep -
Math.PI / 2)));
        return pts;
    }

    private void DrawNodes(Graphics g, PointF[] pts, string[]
labels, Brush bg = null)
    {
        if (bg == null) bg = Brushes.White;
        int r = 15;
        Font f = new Font("Arial", 10);
        StringFormat sf = new StringFormat() { Alignment =
StringAlignment.Center, LineAlignment = StringAlignment.Center };
        for (int i = 0; i < pts.Length; i++)
        {
            RectangleF rect = new RectangleF(pts[i].X - r, pts[i].Y
- r, 2 * r, 2 * r);
            g.FillEllipse(bg, rect);
```

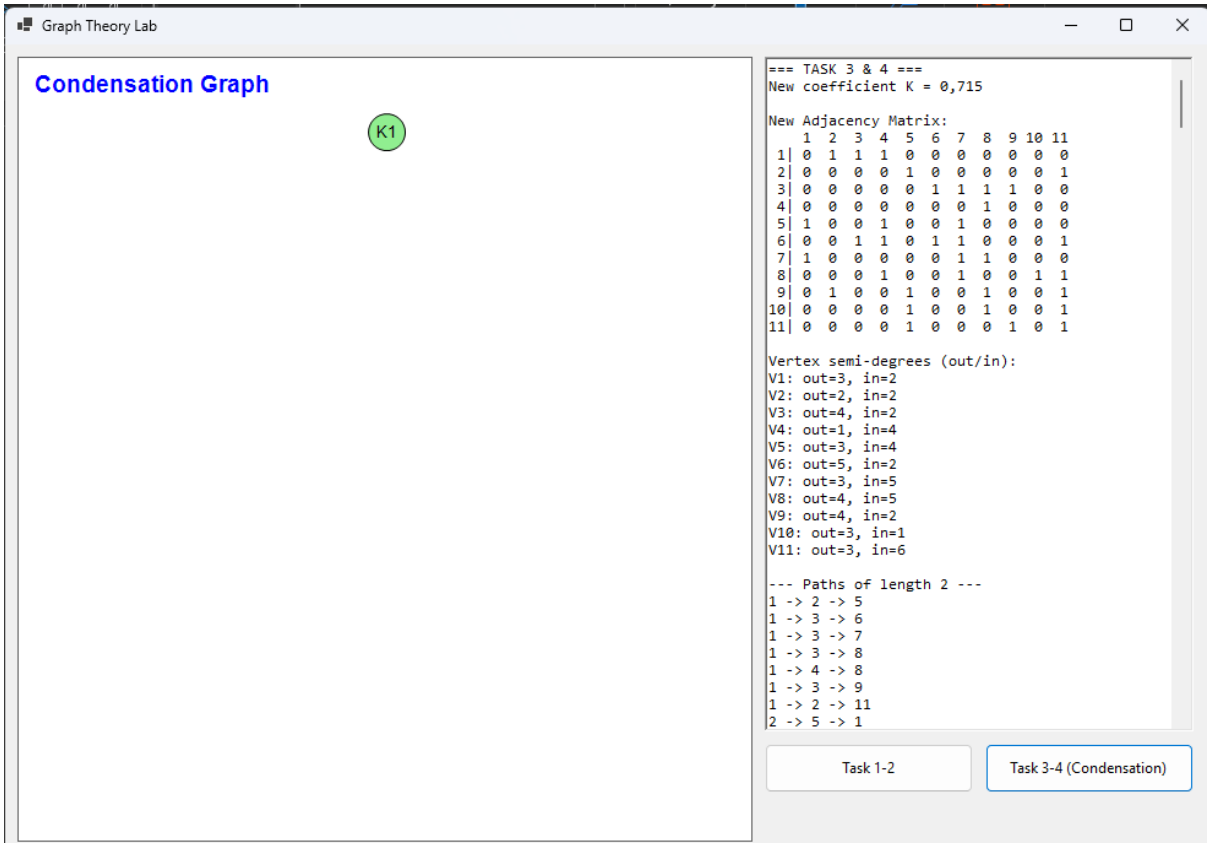
```
            g.DrawEllipse(Pens.Black, rect);
            g.DrawString(labels[i], f, Brushes.Black, rect, sf);
        }
    }

    private void DrawArrow(Graphics g, Pen p, PointF p1, PointF p2)
    {
        float r = 15;
        double angle = Math.Atan2(p2.Y - p1.Y, p2.X - p1.X);
        PointF start = new PointF((float)(p1.X + r *
Math.Cos(angle)), (float)(p1.Y + r * Math.Sin(angle)));
        PointF end = new PointF((float)(p2.X - r * Math.Cos(angle)),
(float)(p2.Y - r * Math.Sin(angle)));
        g.DrawLine(p, start, end);
    }

    private void DrawLoop(Graphics g, PointF p)
    {
        int r = 15;
        g.DrawEllipse(Pens.Black, p.X - r, p.Y - r * 2 - 5, 20, 20);
    }

    private void InitializeComponentCustom()
    {
        this.Size = new Size(1000, 700);
        this.Text = "Graph Theory Lab";
        pbGraph = new PictureBox();
        pbGraph.Location = new Point(10, 10);
        pbGraph.Size = new Size(600, 640);
        pbGraph.BorderStyle = BorderStyle.FixedSingle;
        pbGraph.BackColor = Color.White;
        pbGraph.Paint += pbGraph_Paint;
        this.Controls.Add(pbGraph);
        txtOutput = new RichTextBox();
        txtOutput.Location = new Point(620, 10);
        txtOutput.Size = new Size(350, 550);
        txtOutput.Font = new Font("Consolas", 9);
        this.Controls.Add(txtOutput);
        btnTask1_2 = new Button();
        btnTask1_2.Text = "Task 1-2";
        btnTask1_2.Location = new Point(620, 570);
        btnTask1_2.Size = new Size(170, 40);
```

```
        btnTask1_2.Click += RunTask1_2;
        this.Controls.Add(btnTask1_2);
        btnTask3_4 = new Button();
        btnTask3_4.Text = "Task 3-4 (Condensation)";
        btnTask3_4.Location = new Point(800, 570);
        btnTask3_4.Size = new Size(170, 40);
        btnTask3_4.Click += RunTask3_4;
        this.Controls.Add(btnTask3_4);
    }
}


}
```

Тест програми:




```
--- Graph Characteristics ---
Undirected graph:
V1: deg=4
V2: deg=3
V3: deg=4
V4: deg=3
V5: deg=6
V6: deg=4
V7: deg=6
V8: deg=5
V9: deg=4
V10: deg=3
V11: deg=5
Graph is not regular.

Directed graph:
V1: out=2, in=2
V2: out=2, in=1
V3: out=4, in=0
V4: out=1, in=3
V5: out=3, in=3
V6: out=3, in=2
V7: out=2, in=5
V8: out=3, in=3
V9: out=3, in=2
V10: out=2, in=1
V11: out=2, in=5
Isolated vertices: none
Pendant vertices: none
```


Graph Theory Lab
Condensation Graph
K1
=== TASK 3 & 4 ===
New coefficient K = 0,715
New Adjacency Matrix:
1 2 3 4 5 6 7 8 9 10 11
1| 0 1 1 1 0 0 0 0 0 0 0
2| 0 0 0 0 1 0 0 0 0 0 1
3| 0 0 0 0 0 1 1 1 1 0 0
4| 0 0 0 0 0 0 0 1 0 0 0
5| 1 0 0 1 0 0 1 0 0 0 0
6| 0 0 1 1 0 1 1 0 0 0 1
7| 1 0 0 0 0 0 1 1 0 0 0
8| 0 0 0 1 0 0 1 0 0 1 1
9| 0 1 0 0 1 0 0 1 0 0 1
10| 0 0 0 0 1 0 0 1 0 0 1
11| 0 0 0 0 1 0 0 0 1 0 1
Vertex semi-degrees (out/in):
V1: out=3, in=2
V2: out=2, in=2
V3: out=4, in=2
V4: out=1, in=4
V5: out=3, in=4
V6: out=5, in=2
V7: out=3, in=5
V8: out=4, in=5
V9: out=4, in=2
V10: out=3, in=1
V11: out=3, in=6
--- Paths of length 2 ---
1 -> 2 -> 5
1 -> 3 -> 6
1 -> 3 -> 7
1 -> 3 -> 8
1 -> 4 -> 8
1 -> 3 -> 9
1 -> 2 -> 11
2 -> 5 -> 1
Task 1-2
Task 3-4 (Condensation)

```
1 -> 3 -> 9
1 -> 2 -> 11
2 -> 5 -> 1
2 -> 5 -> 4
2 -> 11 -> 5
2 -> 5 -> 7
2 -> 11 -> 9
2 -> 11 -> 11
3 -> 7 -> 1
3 -> 9 -> 2
3 -> 6 -> 3
3 -> 6 -> 4
3 -> 8 -> 4
3 -> 9 -> 5
3 -> 6 -> 6
3 -> 6 -> 7
3 -> 7 -> 7
3 -> 8 -> 7
3 -> 7 -> 8
3 -> 9 -> 8
3 -> 8 -> 10
3 -> 6 -> 11
3 -> 8 -> 11
3 -> 9 -> 11
4 -> 8 -> 4
4 -> 8 -> 7
4 -> 8 -> 10
4 -> 8 -> 11
5 -> 7 -> 1
5 -> 1 -> 2
5 -> 1 -> 3
5 -> 1 -> 4
5 -> 7 -> 7
5 -> 4 -> 8
5 -> 7 -> 8
6 -> 7 -> 1
6 -> 6 -> 3
6 -> 6 -> 4
6 -> 11 -> 5
```

```
6 -> 7 -> 1
6 -> 6 -> 3
6 -> 6 -> 4
6 -> 11 -> 5
6 -> 3 -> 6
6 -> 6 -> 6
6 -> 3 -> 7
6 -> 6 -> 7
6 -> 7 -> 7
6 -> 3 -> 8
6 -> 4 -> 8
6 -> 7 -> 8
6 -> 3 -> 9
6 -> 11 -> 9
6 -> 6 -> 11
6 -> 11 -> 11
7 -> 7 -> 1
7 -> 1 -> 2
7 -> 1 -> 3
7 -> 1 -> 4
7 -> 8 -> 4
7 -> 7 -> 7
7 -> 8 -> 7
7 -> 7 -> 8
7 -> 8 -> 10
7 -> 8 -> 11
8 -> 7 -> 1
8 -> 10 -> 5
8 -> 11 -> 5
8 -> 7 -> 7
8 -> 4 -> 8
8 -> 7 -> 8
8 -> 10 -> 8
8 -> 11 -> 9
8 -> 10 -> 11
8 -> 11 -> 11
9 -> 5 -> 1
9 -> 5 -> 4
9 -> 8 -> 4
```

```
9 -> 5 -> 4
9 -> 8 -> 4
9 -> 2 -> 5
9 -> 11 -> 5
9 -> 5 -> 7
9 -> 8 -> 7
9 -> 11 -> 9
9 -> 8 -> 10
9 -> 2 -> 11
9 -> 8 -> 11
9 -> 11 -> 11
10 -> 5 -> 1
10 -> 5 -> 4
10 -> 8 -> 4
10 -> 11 -> 5
10 -> 5 -> 7
10 -> 8 -> 7
10 -> 11 -> 9
10 -> 8 -> 10
10 -> 8 -> 11
10 -> 11 -> 11
11 -> 5 -> 1
11 -> 9 -> 2
11 -> 5 -> 4
11 -> 9 -> 5
11 -> 11 -> 5
11 -> 5 -> 7
11 -> 9 -> 8
11 -> 11 -> 9
11 -> 9 -> 11
11 -> 11 -> 11

--- Paths of length 3 ---
1 -> 2 -> 5 -> 1
1 -> 3 -> 7 -> 1
1 -> 3 -> 9 -> 2
1 -> 3 -> 6 -> 3
1 -> 2 -> 5 -> 4
1 -> 3 -> 6 -> 4
```

```
--- Paths of length 3 ---
1 -> 2 -> 5 -> 1
1 -> 3 -> 7 -> 1
1 -> 3 -> 9 -> 2
1 -> 3 -> 6 -> 3
1 -> 2 -> 5 -> 4
1 -> 3 -> 6 -> 4
1 -> 3 -> 8 -> 4
1 -> 4 -> 8 -> 4
1 -> 2 -> 11 -> 5
1 -> 3 -> 9 -> 5
1 -> 3 -> 6 -> 6
1 -> 2 -> 5 -> 7
1 -> 3 -> 6 -> 7
1 -> 3 -> 7 -> 7
1 -> 3 -> 8 -> 7
1 -> 4 -> 8 -> 7
1 -> 3 -> 7 -> 8
1 -> 3 -> 9 -> 8
1 -> 2 -> 11 -> 9
1 -> 3 -> 8 -> 10
1 -> 4 -> 8 -> 10
1 -> 2 -> 11 -> 11
1 -> 3 -> 6 -> 11
1 -> 3 -> 8 -> 11
1 -> 3 -> 9 -> 11
1 -> 4 -> 8 -> 11
2 -> 5 -> 7 -> 1
2 -> 11 -> 5 -> 1
2 -> 5 -> 1 -> 2
2 -> 11 -> 9 -> 2
2 -> 5 -> 1 -> 3
2 -> 5 -> 1 -> 4
2 -> 11 -> 5 -> 4
2 -> 11 -> 9 -> 5
2 -> 11 -> 11 -> 5
2 -> 5 -> 7 -> 7
2 -> 11 -> 5 -> 7
2 -> 5 -> 4 -> 8
```

```
2 -> 11 -> 5 -> 7
2 -> 5 -> 4 -> 8
2 -> 5 -> 7 -> 8
2 -> 11 -> 9 -> 8
2 -> 11 -> 11 -> 9
2 -> 11 -> 9 -> 11
2 -> 11 -> 11 -> 11
3 -> 6 -> 7 -> 1
3 -> 7 -> 7 -> 1
3 -> 8 -> 7 -> 1
3 -> 9 -> 5 -> 1
3 -> 7 -> 1 -> 2
3 -> 6 -> 6 -> 3
3 -> 7 -> 1 -> 3
3 -> 6 -> 6 -> 4
3 -> 7 -> 1 -> 4
3 -> 7 -> 8 -> 4
3 -> 9 -> 5 -> 4
3 -> 9 -> 8 -> 4
3 -> 6 -> 11 -> 5
3 -> 8 -> 10 -> 5
3 -> 8 -> 11 -> 5
3 -> 9 -> 2 -> 5
3 -> 9 -> 11 -> 5
3 -> 6 -> 3 -> 6
3 -> 6 -> 6 -> 6
3 -> 6 -> 3 -> 7
3 -> 6 -> 6 -> 7
3 -> 6 -> 7 -> 7
3 -> 7 -> 7 -> 7
3 -> 7 -> 8 -> 7
3 -> 8 -> 7 -> 7
3 -> 9 -> 5 -> 7
3 -> 9 -> 8 -> 7
3 -> 6 -> 3 -> 8
3 -> 6 -> 4 -> 8
3 -> 6 -> 7 -> 8
3 -> 7 -> 7 -> 8
3 -> 8 -> 4 -> 8
```

```
3 -> 6 -> 7 -> 8
3 -> 7 -> 7 -> 8
3 -> 8 -> 4 -> 8
3 -> 8 -> 7 -> 8
3 -> 8 -> 10 -> 8
3 -> 6 -> 3 -> 9
3 -> 6 -> 11 -> 9
3 -> 8 -> 11 -> 9
3 -> 9 -> 11 -> 9
3 -> 7 -> 8 -> 10
3 -> 9 -> 8 -> 10
3 -> 6 -> 6 -> 11
3 -> 6 -> 11 -> 11
3 -> 7 -> 8 -> 11
3 -> 8 -> 10 -> 11
3 -> 8 -> 11 -> 11
3 -> 9 -> 2 -> 11
3 -> 9 -> 8 -> 11
3 -> 9 -> 11 -> 11
4 -> 8 -> 7 -> 1
4 -> 8 -> 10 -> 5
4 -> 8 -> 11 -> 5
4 -> 8 -> 7 -> 7
4 -> 8 -> 4 -> 8
4 -> 8 -> 7 -> 8
4 -> 8 -> 10 -> 8
4 -> 8 -> 11 -> 9
4 -> 8 -> 10 -> 11
4 -> 8 -> 11 -> 11
5 -> 7 -> 7 -> 1
5 -> 7 -> 1 -> 2
5 -> 7 -> 1 -> 3
5 -> 4 -> 8 -> 4
5 -> 7 -> 1 -> 4
5 -> 7 -> 8 -> 4
5 -> 1 -> 2 -> 5
5 -> 1 -> 3 -> 6
5 -> 1 -> 3 -> 7
5 -> 4 -> 8 -> 7
```

```
5 -> 1 -> 4 -> 8
5 -> 7 -> 7 -> 8
5 -> 1 -> 3 -> 9
5 -> 4 -> 8 -> 10
5 -> 7 -> 8 -> 10
5 -> 1 -> 2 -> 11
5 -> 4 -> 8 -> 11
5 -> 7 -> 8 -> 11
6 -> 3 -> 7 -> 1
6 -> 6 -> 7 -> 1
6 -> 7 -> 7 -> 1
6 -> 11 -> 5 -> 1
6 -> 3 -> 9 -> 2
6 -> 7 -> 1 -> 2
6 -> 11 -> 9 -> 2
6 -> 3 -> 6 -> 3
6 -> 6 -> 6 -> 3
6 -> 7 -> 1 -> 3
6 -> 3 -> 6 -> 4
6 -> 3 -> 8 -> 4
6 -> 4 -> 8 -> 4
6 -> 6 -> 6 -> 4
6 -> 7 -> 1 -> 4
6 -> 7 -> 8 -> 4
6 -> 11 -> 5 -> 4
6 -> 3 -> 9 -> 5
6 -> 6 -> 11 -> 5
6 -> 11 -> 9 -> 5
6 -> 11 -> 11 -> 5
6 -> 3 -> 6 -> 6
6 -> 6 -> 3 -> 6
6 -> 6 -> 6 -> 6
6 -> 3 -> 6 -> 7
6 -> 3 -> 7 -> 7
6 -> 3 -> 8 -> 7
6 -> 4 -> 8 -> 7
6 -> 6 -> 3 -> 7
6 -> 6 -> 6 -> 7
6 -> 6 -> 7 -> 7
```

```
6 -> 7 -> 7 -> 7
6 -> 7 -> 8 -> 7
6 -> 11 -> 5 -> 7
6 -> 3 -> 7 -> 8
6 -> 3 -> 9 -> 8
6 -> 6 -> 3 -> 8
6 -> 6 -> 4 -> 8
6 -> 6 -> 7 -> 8
6 -> 7 -> 7 -> 8
6 -> 11 -> 9 -> 8
6 -> 6 -> 3 -> 9
6 -> 6 -> 11 -> 9
6 -> 11 -> 11 -> 9
6 -> 3 -> 8 -> 10
6 -> 4 -> 8 -> 10
6 -> 7 -> 8 -> 10
6 -> 3 -> 6 -> 11
6 -> 3 -> 8 -> 11
6 -> 3 -> 9 -> 11
6 -> 4 -> 8 -> 11
6 -> 6 -> 6 -> 11
6 -> 6 -> 11 -> 11
6 -> 7 -> 8 -> 11
6 -> 11 -> 9 -> 11
6 -> 11 -> 11 -> 11
7 -> 7 -> 7 -> 1
7 -> 8 -> 7 -> 1
7 -> 7 -> 1 -> 2
7 -> 7 -> 1 -> 3
7 -> 7 -> 1 -> 4
7 -> 7 -> 8 -> 4
7 -> 1 -> 2 -> 5
7 -> 8 -> 10 -> 5
7 -> 8 -> 11 -> 5
7 -> 1 -> 3 -> 6
7 -> 1 -> 3 -> 7
7 -> 7 -> 7 -> 7
7 -> 7 -> 8 -> 7
7 -> 8 -> 7 -> 7
```

```
7 -> 8 -> 11 -> 5
7 -> 1 -> 3 -> 6
7 -> 1 -> 3 -> 7
7 -> 7 -> 7 -> 7
7 -> 7 -> 8 -> 7
7 -> 8 -> 7 -> 7
7 -> 1 -> 3 -> 8
7 -> 1 -> 4 -> 8
7 -> 7 -> 7 -> 8
7 -> 8 -> 4 -> 8
7 -> 8 -> 7 -> 8
7 -> 8 -> 10 -> 8
7 -> 1 -> 3 -> 9
7 -> 8 -> 11 -> 9
7 -> 7 -> 8 -> 10
7 -> 1 -> 2 -> 11
7 -> 7 -> 8 -> 11
7 -> 8 -> 10 -> 11
7 -> 8 -> 11 -> 11
8 -> 7 -> 7 -> 1
8 -> 10 -> 5 -> 1
8 -> 11 -> 5 -> 1
8 -> 7 -> 1 -> 2
8 -> 11 -> 9 -> 2
8 -> 7 -> 1 -> 3
8 -> 4 -> 8 -> 4
8 -> 7 -> 1 -> 4
8 -> 7 -> 8 -> 4
8 -> 10 -> 5 -> 4
8 -> 10 -> 8 -> 4
8 -> 11 -> 5 -> 4
8 -> 10 -> 11 -> 5
8 -> 11 -> 9 -> 5
8 -> 11 -> 11 -> 5
8 -> 4 -> 8 -> 7
8 -> 7 -> 7 -> 7
8 -> 7 -> 8 -> 7
8 -> 10 -> 5 -> 7
8 -> 10 -> 8 -> 7
```

```
8 -> 11 -> 5 -> 7
8 -> 7 -> 7 -> 8
8 -> 11 -> 9 -> 8
8 -> 10 -> 11 -> 9
8 -> 11 -> 11 -> 9
8 -> 4 -> 8 -> 10
8 -> 7 -> 8 -> 10
8 -> 10 -> 8 -> 10
8 -> 4 -> 8 -> 11
8 -> 7 -> 8 -> 11
8 -> 10 -> 8 -> 11
8 -> 10 -> 11 -> 11
8 -> 11 -> 9 -> 11
8 -> 11 -> 11 -> 11
9 -> 2 -> 5 -> 1
9 -> 5 -> 7 -> 1
9 -> 8 -> 7 -> 1
9 -> 11 -> 5 -> 1
9 -> 5 -> 1 -> 2
9 -> 11 -> 9 -> 2
9 -> 5 -> 1 -> 3
9 -> 2 -> 5 -> 4
9 -> 5 -> 1 -> 4
9 -> 11 -> 5 -> 4
9 -> 2 -> 11 -> 5
9 -> 8 -> 10 -> 5
9 -> 8 -> 11 -> 5
9 -> 11 -> 9 -> 5
9 -> 11 -> 11 -> 5
9 -> 2 -> 5 -> 7
9 -> 5 -> 7 -> 7
9 -> 8 -> 7 -> 7
9 -> 11 -> 5 -> 7
9 -> 5 -> 4 -> 8
9 -> 5 -> 7 -> 8
9 -> 8 -> 4 -> 8
9 -> 8 -> 7 -> 8
9 -> 8 -> 10 -> 8
9 -> 11 -> 9 -> 8
```

```
9 -> 2 -> 11 -> 9
9 -> 8 -> 11 -> 9
9 -> 11 -> 11 -> 9
9 -> 2 -> 11 -> 11
9 -> 8 -> 10 -> 11
9 -> 8 -> 11 -> 11
9 -> 11 -> 9 -> 11
9 -> 11 -> 11 -> 11
10 -> 5 -> 7 -> 1
10 -> 8 -> 7 -> 1
10 -> 11 -> 5 -> 1
10 -> 5 -> 1 -> 2
10 -> 11 -> 9 -> 2
10 -> 5 -> 1 -> 3
10 -> 5 -> 1 -> 4
10 -> 11 -> 5 -> 4
10 -> 8 -> 10 -> 5
10 -> 8 -> 11 -> 5
10 -> 11 -> 9 -> 5
10 -> 11 -> 11 -> 5
10 -> 5 -> 7 -> 7
10 -> 8 -> 7 -> 7
10 -> 11 -> 5 -> 7
10 -> 5 -> 4 -> 8
10 -> 5 -> 7 -> 8
10 -> 8 -> 4 -> 8
10 -> 8 -> 7 -> 8
10 -> 8 -> 10 -> 8
10 -> 11 -> 9 -> 8
10 -> 8 -> 11 -> 9
10 -> 11 -> 11 -> 9
10 -> 8 -> 10 -> 11
10 -> 8 -> 11 -> 11
10 -> 11 -> 9 -> 11
10 -> 11 -> 11 -> 11
11 -> 5 -> 7 -> 1
11 -> 9 -> 5 -> 1
11 -> 11 -> 5 -> 1
11 -> 5 -> 1 -> 2
```

```
11 -> 9 -> 5 -> 1
11 -> 11 -> 5 -> 1
11 -> 5 -> 1 -> 2
11 -> 11 -> 9 -> 2
11 -> 5 -> 1 -> 3
11 -> 5 -> 1 -> 4
11 -> 9 -> 5 -> 4
11 -> 9 -> 8 -> 4
11 -> 11 -> 5 -> 4
11 -> 9 -> 2 -> 5
11 -> 9 -> 11 -> 5
11 -> 11 -> 9 -> 5
11 -> 11 -> 11 -> 5
11 -> 5 -> 7 -> 7
11 -> 9 -> 5 -> 7
11 -> 9 -> 8 -> 7
11 -> 11 -> 5 -> 7
11 -> 5 -> 4 -> 8
11 -> 5 -> 7 -> 8
11 -> 11 -> 9 -> 8
11 -> 9 -> 11 -> 9
11 -> 11 -> 11 -> 9
11 -> 9 -> 8 -> 10
11 -> 9 -> 2 -> 11
11 -> 9 -> 8 -> 11
11 -> 9 -> 11 -> 11
11 -> 11 -> 9 -> 11
11 -> 11 -> 11 -> 11
```

```
Reachability Matrix:
    1  2  3  4  5  6  7  8  9 10 11
 1| 1  1  1  1  1  1  1  1  1  1  1
 2| 1  1  1  1  1  1  1  1  1  1  1
 3| 1  1  1  1  1  1  1  1  1  1  1
 4| 1  1  1  1  1  1  1  1  1  1  1
 5| 1  1  1  1  1  1  1  1  1  1  1
 6| 1  1  1  1  1  1  1  1  1  1  1
 7| 1  1  1  1  1  1  1  1  1  1  1
 8| 1  1  1  1  1  1  1  1  1  1  1
 9| 1  1  1  1  1  1  1  1  1  1  1
10| 1  1  1  1  1  1  1  1  1  1  1
11| 1  1  1  1  1  1  1  1  1  1  1

Strong Connectivity Matrix:
    1  2  3  4  5  6  7  8  9 10 11
 1| 1  1  1  1  1  1  1  1  1  1  1
 2| 1  1  1  1  1  1  1  1  1  1  1
 3| 1  1  1  1  1  1  1  1  1  1  1
 4| 1  1  1  1  1  1  1  1  1  1  1
 5| 1  1  1  1  1  1  1  1  1  1  1
 6| 1  1  1  1  1  1  1  1  1  1  1
 7| 1  1  1  1  1  1  1  1  1  1  1
 8| 1  1  1  1  1  1  1  1  1  1  1
 9| 1  1  1  1  1  1  1  1  1  1  1
10| 1  1  1  1  1  1  1  1  1  1  1
11| 1  1  1  1  1  1  1  1  1  1  1

Strongly Connected Components:
K1: { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 }

Condensation Graph Matrix:
    1
 1| 0
```

Висновок: У ході лабораторної роботи було досліджено ключові характеристики графів, включаючи степені вершин, шляхи та критерії зв'язності. Практично реалізовано метод транзитивного замикання (алгоритм Уоршелла), що дозволив ефективно побудувати матриці досяжності та сильної зв'язності. На основі отриманих даних було побудовано граф конденсації, що дало змогу структурувати складний орграф та виділити в ньому компоненти сильної зв'язності. Отримані результати підтвердили теоретичні знання про поведінку динамічних зв'язкових структур та закріпили навички програмної реалізації алгоритмів теорії графів.